

Suggested List of Laboratory Experiments/Assignments

1	Introduction to Database Creation
	Objective: Set up and create a database using MySQL/PostgreSQL. Task: Entities and Relationships: • Student (StudentID, Name, Email, Age, Address) • Course (CourseID, CourseName, Credits, InstructorID) • Instructor (InstructorID, Name, Email, Department) • Enrollment (EnrollmentID, StudentID, CourseID, EnrollmentDate) Relationships: • A student can enroll in multiple courses. • A course can have multiple students. • An instructor teaches one or more courses.

2	Views and Indexing
	Optimizing a Student Course Management System with Views and Indexing for Faster Query Performance. Objective: Understand views and indexing for performance optimization. Task: • Create views for easy reporting on students and course details. • Update and delete records using views where applicable. • Use indexing to speed up searches and measure query performance improvements.

3

SQL Queries with Filters and Sorting

Advanced Querying in a Student Course Management System Using Filters, Sorting, and Nested Subqueries

Objective: Use SQL filters and sorting clauses.

Task:

- Filtering Data with WHERE (Find students above a certain age, filter by instructor)
- Sorting with ORDER BY (Alphabetically, by credits)
- Aggregation with GROUP BY (Student count per course, average age)
- Nested Subqueries for Complex Queries

4

Multi-Table Joins

Develop a Customer Order Management System to Retrieve Data Using SQL Joins

Objective: Learn inner and outer joins in SQL.

Task:

- INNER JOIN: Retrieve all customer orders (only customers who have placed orders).
- LEFT JOIN: Retrieve all customers, including those who haven't placed any orders.
- RIGHT JOIN: Retrieve all orders, including those placed by customers not in the Customers table.
- FULL OUTER JOIN: Retrieve a complete list of customers and orders, including those that do not have a match in the other table.

5

PL/SQL Procedures and Functions

Design and Implement a Sales Management System with Stored Procedures and Functions for Efficient Data Handling

Objective: Implement stored procedures and functions.

Task:

- Create a stored procedure to add data to a table.
- Create a function to calculate and return total sales.

6	Triggers and Cursors Develop an Employee Management System with Triggers for Audit Logging and Cursors for Efficient Data Processing Objective: Implement triggers and cursors in PL/SQL. Task: • Create a trigger on the Employees table that logs changes (INSERT, UPDATE, DELETE) into the Employee_Audit table. • Use a cursor to iterate through the Employees table and process salary increments based on certain conditions (e.g., increase salary by 10% for employees in the IT department).
7	NoSQL Database Introduction – MongoDB Aggregation and Indexing Objectives: Work with MongoDB for NoSQL data storage. Use MongoDB aggregation and indexing for data analysis. Task: • Create a MongoDB collection for user data. • Perform CRUD (Create, Read, Update, Delete) operations on the collection. • Write aggregation queries to calculate statistics (e.g., total sales, average age). • Create indexes and observe performance improvements.
8	Case Study – Social Media Data Processing using NoSQL Objective: Process unstructured social media data using NoSQL databases. Task: • Import a dataset with social media posts into MongoDB. • Write queries to filter and analyze data (e.g., find top trending hashtags).
9	MySQL Database Backup and Recovery Objective: To understand and practice the process of taking a backup of a MySQL database to ensure data security and disaster recovery.

10	Compulsory Mini Project (Group of 2 to 3 students can develop mini project) These projects align well with the syllabus, incorporating concepts like relational database design, SQL queries, PL/SQL, transaction management, and NoSQL. They also offer practical applications that can be demonstrated easily. Database connectivity with frontend and backend is compulsory.
	1. Library Management System Objective: Design a relational database to manage books, users, loans, and overdue notifications. Features: • Book search with filters • User login and role-based access • Loan and return history management
	2. Online Retail Store Database Objective: Develop a database for an e-commerce store to manage products, customers, orders, and inventory. Features: • Order placement and status tracking • Real-time inventory updates • Customer order history
	3. Student Attendance and Grade Management System Objective: Create a database for managing student attendance and grades for courses. Features: • Attendance recording and reporting • Grade calculation and storage • Performance analysis for students
	4. Hospital Patient Management System Objective: Design a database for managing patients, doctors, and appointments. Features: • Patient registration and medical history tracking • Doctor schedules and appointment booking • Billing and prescription management

	5. Movie Ticket Booking System Objective: Create a database for booking movie tickets with seat selection. Features: • Show listings and seat availability • Ticket booking and cancellation • Customer feedback collection
	6. Food Delivery System Objective: Design a database for managing restaurants, orders, and deliveries. Features: • Restaurant menu and order placement • Customer and delivery person management • Order tracking and delivery status
	7. Real Estate Property Management System Objective: Develop a system to manage properties, agents, and customers. Features: • Property listing with filters • Customer inquiries and appointments • Agent performance tracking
	8. Social Media Data Analysis using MongoDB Objective: Use a NoSQL database to analyze social media data such as user posts and hashtags. Features: • Hashtag trend identification • Sentiment analysis on user comments • Data aggregation for user engagement
	9. Online Voting System Objective: Create a secure database for managing elections and votes. Features: • Voter registration and verification • Voting and vote tallying • Result announcement and audit trail

10. Event Management System

Objective: Develop a database to manage events, participants, and schedules. Features:

- Event registration and attendee tracking
- Event schedules and notifications
-

Feedback collection and reporting

Assignment-1

Title:

Introduction to Database Creation using MySQL/PostgreSQL.

Objectives:

1. To design a simple academic database using relational database concepts.
2. To identify entities, attributes, and relationships.
3. To create tables using SQL with primary keys and foreign keys.
4. To understand many-to-many and one-to-many relationships.
5. To implement an ER diagram and convert it into database tables.

Problem Statement:

Design and implement a database for an academic system containing **Students, Courses, Instructors,** and **Enrollments.**

The system should satisfy the following:

A student can enroll in multiple courses.
A course can have multiple students.
An instructor teaches one or more courses.

Create tables, apply appropriate constraints, insert sample data, and execute simple queries.

Theory Concept:

1. Database

A structured collection of data stored and managed for efficient retrieval, insertion, and modification.

2. DBMS

A Database Management System that allows creation, modification, and control of databases.

3. Entity

A real-world object with attributes.

Examples: **Student, Course, Instructor, Enrollment.**

Types of entities:

A collection of a similar kind of entities is called an entity set or entity type. It is of two types.

1. Weak entity set: Weak entity set does not contain any key attribute (i.e. primary key). It is

denoted by



2. Strong entity set: The entity type containing a key attribute are called strong entity or regular entity set. It is denoted by



e.g.- The 'STUDENT' entity has a key attribute ROLL NO which uniquely identifies it.

4. Attribute

Properties of an entity.

Example: Student → Name, Email, Age.

5. Primary Key (PK)

A unique identifier for each record.

Example: StudentID.

6. Foreign Key (FK)

A field that links two tables.

Example: Course.InstructorID → Instructor.InstructorID.

7. Cardinality

Cardinality specifies the no of instances of an entity associated with another entity, participating in a relationship. Based on the cardinality binary relationship can be further classified into the following categories.

a>one----to---one

b>one---to---many

c>many---to---one

d>many---to---many

8. ER Diagram

A diagrammatic representation of entities and their relationships.

9. SQL (Structured Query Language) is the standard language used to communicate with a Relational Database Management System (RDBMS). It is used to create, manage, retrieve, and manipulate data stored in relational databases.

In DBMS, data is stored in the form of tables (relations) consisting of rows (tuples) and columns (attributes). SQL allows users, developers, and administrators to interact with these tables efficiently.

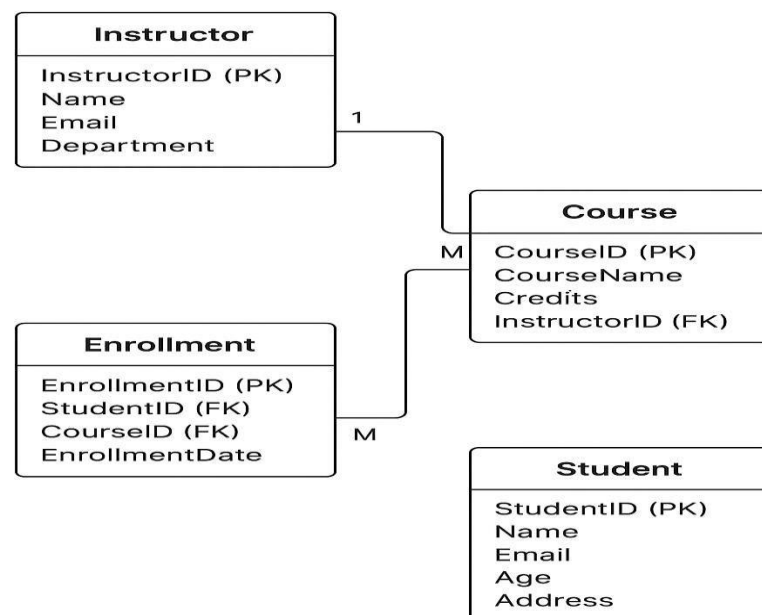


Fig 1:ER Diagram

SQL Commands

1.1 Create Database

MySQL:

```
CREATE DATABASE AcademicDB;  
USE AcademicDB;
```

1.2 Create Tables

Student Table

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,
```

```
Name VARCHAR(100),
Email VARCHAR(100) UNIQUE,
Age INT,
Address VARCHAR(200)
);
```

Instructor Table:

```
CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100) UNIQUE,
    Department VARCHAR(100)
);
```

Course Table

```
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100),
    Credits INT,
    InstructorID INT,
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)
);
```

Enrollment Table

```
CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

1.3 Insert Data into table:

```
INSERT INTO Student VALUES
(1,'Rahul','rahul@gmail.com',20,'Delhi'),
(2,'Anita','anita@gmail.com',21,'Mumbai');
```

```
INSERT INTO Instructor VALUES
```

```
(101,'Dr. Mehta','mehta@college.com','Computer Science'),  
(102,'Prof. Rao','rao@college.com','Mathematics');
```

```
INSERT INTO Course VALUES  
(501,'DBMS',4,101),  
(502,'Calculus',3,102);
```

```
INSERT INTO Enrollment VALUES  
(1001,1,501,'2024-07-01'),  
(1002,1,502,'2024-07-05'),  
(1003,2,501,'2024-07-10');
```

```
List all students  
SELECT * FROM Student;
```

```
List all courses taught by a specific instructor  
SELECT CourseName  
FROM Course  
WHERE InstructorID = 101;
```

```
Find all courses a student is enrolled in  
SELECT Student.Name, Course.CourseName  
FROM Enrollment  
JOIN Student ON Enrollment.StudentID = Student.StudentID  
JOIN Course ON Enrollment.CourseID = Course.CourseID  
WHERE Student.StudentID = 1;
```

Output

Students must attach screenshots of:

1. Database creation
2. Table creation
3. Sample insert operations
4. Select queries execution

Conclusion:

Thus, a **Student Course Management System database** was successfully created using **MySQL**.

The experiment demonstrated how to create a database, define tables, apply primary keys and foreign keys, and establish relationships between multiple tables.

Assignment-2

Title: Optimizing a Student Course Management System with Views and Indexing for Faster Query Performance.

1. Aim

To create and use SQL **views** and **indexes** for efficient data retrieval and to improve query performance in a Student Course Management System.

2. Objectives

- To understand the concept and purpose of SQL **views**.
- To learn how to perform **data reporting** using views.
- To update and delete records using updatable views.
- To apply **indexes** to improve query execution speed.
- To measure and compare query performance **before and after indexing**.

-
- Understand when and where indexing should be used.
 - Write optimized SQL queries for large datasets.

3. Theory

3.1 Views

In SQL, a view is a virtual table based on the result-set of an SQL statement. A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database. You can add SQL functions, WHERE, and JOIN statements to a view and present the data as if the data were coming from one single table. A view is a virtual table, which consists of a set of columns from one or more tables. It is similar to a table but it does not store in the database. View is a query stored as an object.

Syntax: CREATE VIEW AS SELECT FROM relation_name WHERE (Condition)

Example:

```
SQL> CREATE VIEW employee AS SELECT empno,ename,job FROM EMP WHERE job = 'clerk';
```

```
SQL> View created.
```

Advantages of Views:

- Simplifies complex queries.
- Provides security by restricting direct table access.
- Useful for reporting and analytics.

Types of Views:

- **Simple View**
 - **Complex View**
 - **Materialized View**
-

3.2 Indexing

An **index** is a database object that improves the speed of data retrieval. An index is an ordered set of pointers to the data in a table. It is based on the data values in one or more columns of the table. SQL Base stores indexes separately from tables. An index provides two benefits: It improves performance because it makes data access faster. It ensures uniqueness. A table with a unique index cannot have two rows with the same values in the column or columns that form the index key.

Syntax:

CREATE INDEX on (attrib1,attrib 2....attrib n);

Example: CREATE INDEX id1 on emp(empno,dept_no);

Types of Indexes:

- **Primary index**
- **Unique index**
- **Composite index**
- **Full-text index** (DB-dependent)

Advantages:

- Faster SELECT queries
- Efficient searching, filtering, sorting

Disadvantages:

- Slower INSERT, UPDATE, DELETE
- Requires extra storage

4. Tasks

1. Create views for easy reporting of **student-course-instructor** relationships.
 2. Update and delete records using views where allowed.
-

-
3. Create indexes on frequently searched fields.
 4. Compare query performance **before and after indexing**.

5. Program / SQL

5.1 Creating Base Tables

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(100),  
    Age INT,  
    Address VARCHAR(200)  
);
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(100),  
    Credits INT,  
    InstructorID INT  
);
```

```
CREATE TABLE Instructor (  
    InstructorID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(100),  
    Department VARCHAR(100)  
);
```

```
CREATE TABLE Enrollment (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    EnrollmentDate DATE,  
    FOREIGN KEY(StudentID) REFERENCES Student(StudentID),
```

```
FOREIGN KEY (CourseID) REFERENCES Course (CourseID)
);
```

5.2 Creating a View for Student-Course Report

```
CREATE VIEW StudentCourseView AS
SELECT
    S.StudentID, S.Name AS StudentName,
    C.CourseName, C.Credits,
    I.Name AS InstructorName
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID
JOIN Instructor I ON C.InstructorID = I.InstructorID;
```

5.3 Viewing the Report

```
SELECT * FROM StudentCourseView;
```

5.4 Updating Records Using a View (If Updatable)

```
UPDATE StudentCourseView
SET Credits = 5
WHERE CourseName = 'Data Structures';
```

5.5 Creating Indexes

```
CREATE INDEX idx_student_name ON Student (Name);
CREATE INDEX idx_course_name ON Course (CourseName);
CREATE INDEX idx_enrollment_student ON Enrollment (StudentID);
```

5.6 Checking Query Performance

Before Indexing:

```
EXPLAIN SELECT * FROM Student WHERE Name = 'John';
```

After Indexing:

```
EXPLAIN SELECT * FROM Student WHERE Name = 'John';
```

1. Display total number of students enrolled in each course.

```
SELECT  
  
    C.CourseName,  
  
    COUNT(E.StudentID) AS Total_Students  
  
FROM Course C  
  
LEFT JOIN Enrollment E ON C.CourseID = E.CourseID  
  
GROUP BY C.CourseName;
```

2. Display youngest student enrolled under each instructor.

```
SELECT  
  
    I.InstructorID,  
  
    I.Name AS InstructorName,  
  
    S.StudentID,  
  
    S.Name AS StudentName,  
  
    S.Age  
  
FROM Instructor I  
  
JOIN Course C ON I.InstructorID = C.InstructorID  
  
JOIN Enrollment E ON C.CourseID = E.CourseID  
  
JOIN Student S ON S.StudentID = E.StudentID
```

```
WHERE S.Age = (  
    SELECT MIN(S2.Age)  
    FROM Course C2  
    JOIN Enrollment E2 ON C2.CourseID = E2.CourseID  
    JOIN Student S2 ON S2.StudentID = E2.StudentID   WHERE C2.InstructorID = I.InstructorID);
```

3. Display number of courses taught in each department with department name.

```
SELECT  
  
    I.Department,  
  
    COUNT(C.CourseID) AS Total_Courses  
  
FROM Instructor I  
  
LEFT JOIN Course C ON I.InstructorID = C.InstructorID  
  
GROUP BY I.Department;
```

4. Display student details sorted by age in increasing order.

```
SELECT  
  
    StudentID, Name, Email, Age, Address  
  
FROM Student  
  
ORDER BY Age ASC;
```

5. Show records of students older than 20 in each course.

```
SELECT  
  
    S.StudentID, S.Name, S.Age,  
  
    C.CourseName
```

FROM Student S

JOIN Enrollment E ON S.StudentID = E.StudentID

JOIN Course C ON C.CourseID = E.CourseID

WHERE S.Age > 20

ORDER BY C.CourseName;

6. Display students along with the courses they are enrolled in.
7. Find total enrollments done per month.
8. Display instructors teaching more than 2 courses.
9. List students whose name starts with letter 'A'.

CONCLUSION

Thus, the Student Course Management System database was effectively optimized using MySQL. The experiment illustrated the creation and use of views to simplify complex queries and support reporting, along with the implementation of indexing to enhance query performance. It also involved comparing execution efficiency before and after indexing, emphasizing the importance of optimization techniques for handling large datasets efficiently.

Assignment-3

Title: Advanced Querying in a Student Course Management System Using Filters, Sorting, Aggregation & Nested Subqueries.

1. Aim

To apply SQL filtering, sorting, grouping, and nested subqueries to retrieve meaningful insights from the Student Course Management System.

2. Objectives

- To understand and use the **WHERE** clause for filtering data.
- To perform sorting using **ORDER BY**.
- To apply **GROUP BY** for aggregations.
- To use **nested subqueries** to solve complex retrieval tasks.
- To analyze student, course, and instructor data effectively.

3. Theory

3.1 WHERE Clause (Filtering)

The **WHERE** clause is used to filter records based on conditions.

Examples:

- Filter students above a certain age
- Filter courses taught by a specific instructor

Syntax:

```
SELECT columns FROM table WHERE condition;
```

3.2 ORDER BY Clause (Sorting)

Used to sort data in **ascending (ASC)** or **descending (DESC)** order.

Examples:

- Sort students alphabetically
- Sort courses by credits

Syntax:

```
SELECT columns FROM table ORDER BY column_name ASC|DESC;
```

3.3 GROUP BY Clause (Aggregation)

Used to group rows and apply aggregate functions such as:

- COUNT()
- AVG()
- MAX()
- MIN()

Syntax:

```
SELECT column, COUNT(*) FROM table GROUP BY column;
```

3.4 Nested Subqueries

Queries inside another query to solve complex logic.

Example uses:

- Find students enrolled in more than 2 courses
- Find highest/lowest age
- Compare aggregated values

Syntax:

```
SELECT * FROM table WHERE column = (SELECT value FROM table);
```

4. Tasks

1. Filter students above a certain age.
 2. Filter courses taught by a specific instructor.
 3. Sort students alphabetically.
 4. Sort courses by credits.
 5. Count number of students per course.
 6. Calculate average student age.
 7. Use nested subqueries for complex scenarios.
-

5. Program / SQL

5.1 Filtering Data with WHERE

a) Find students above age 20

```
SELECT StudentID, Name, Age  
FROM Student  
WHERE Age > 20;
```

b) Filter courses taught by instructor 'Dr. Sharma'

```
SELECT C.CourseID, C.CourseName
FROM Course C
JOIN Instructor I ON C.InstructorID = I.InstructorID
WHERE I.Name = 'Dr. Sharma';
```

5.2 Sorting with ORDER BY

a) Sort students alphabetically

```
SELECT StudentID, Name, Age
FROM Student
ORDER BY Name ASC;
```

b) Sort courses by credits (descending)

```
SELECT CourseID, CourseName, Credits
FROM Course
ORDER BY Credits DESC;
```

5.3 Aggregation using GROUP BY

a) Student count per course

```
SELECT
    C.CourseName,
    COUNT(E.StudentID) AS Total_Students
FROM Course C
LEFT JOIN Enrollment E ON C.CourseID = E.CourseID
GROUP BY C.CourseName;
```

b) Average age of students

```
SELECT AVG(Age) AS Average_Student_Age
FROM Student;
```

5.4 Nested Subqueries

a) Display students older than the average student age

```
SELECT *  
FROM Student  
WHERE Age > (SELECT AVG(Age) FROM Student);
```

b) Display courses having more enrollments than the average number of enrollments

```
SELECT C.CourseName  
FROM Course C  
JOIN Enrollment E ON C.CourseID = E.CourseID  
GROUP BY C.CourseName  
HAVING COUNT(E.StudentID) >  
(  
    SELECT AVG(Cnt)  
    FROM (  
        SELECT COUNT(StudentID) AS Cnt  
        FROM Enrollment  
        GROUP BY CourseID  
    ) AS AvgTable  
);
```

6. Output

- Successfully executed filtering using **WHERE** clause.
- Sorted data using **ORDER BY**.
- Performed grouping and aggregations using **GROUP BY**.
- Executed nested subqueries for advanced operations.
- Retrieved meaningful reports from the Student Course Management System.

Conclusion:

This experiment demonstrated how SQL filtering, sorting, aggregation, and nested subqueries help retrieve meaningful and organized information from the Student Course Management System. It strengthened the understanding of advanced query techniques for efficient data analysis.

Assignment-4

Title: Develop a Customer Order Management System to Retrieve Data Using SQL Joins

1. Aim

To understand and implement various SQL joins (INNER, LEFT, RIGHT, FULL OUTER) to retrieve combined records from multiple related tables.

2. Objectives

- To learn the purpose and application of **SQL joins**.
- To retrieve data involving multiple tables using **different join types**.
- To understand the difference between matched and unmatched records.
- To analyze customer and order relationships using SQL.

3. Theory

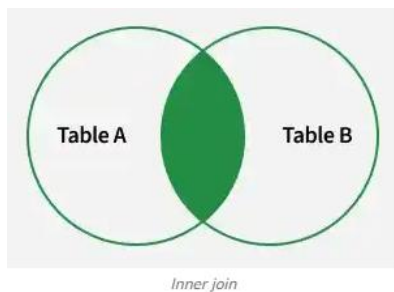
3.1 What Are SQL Joins?

SQL Joins allow retrieval of data from two or more related tables using a common field (usually a primary key–foreign key relationship).

3.2 Types of Joins

a) INNER JOIN

Returns only matching records from both tables.

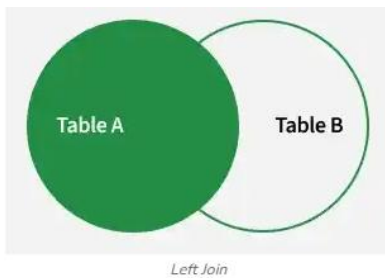


Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,... FROM table1 INNER  
JOIN  
table2 ON table1.matching_column = table2.matching_column;
```

b) LEFT JOIN (LEFT OUTER JOIN)

Returns all records from the left table and matched records from the right table.

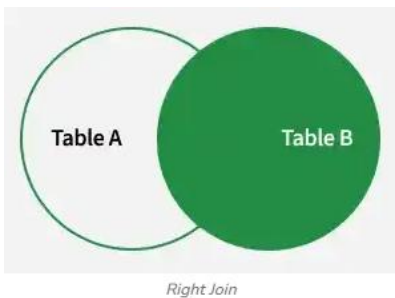


Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,...  
FROM table1  
LEFT JOIN table2  
ON table1.matching_column = table2.matching_column;
```

c) RIGHT JOIN (RIGHT OUTER JOIN)

Returns all records from the right table and matched records from the left table.



Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....  
FROM table1  
RIGHT JOIN table2  
ON table1.matching_column = table2.matching_column;
```

d) FULL OUTER JOIN

Returns all records from both tables, matched and unmatched.



Full Join

```
SELECT table1.column1,table1.column2,table2.column1,....  
FROM table1  
FULL JOIN table2  
ON table1.matching_column = table2.matching_column;
```

4. Tasks

1. Retrieve all customer orders using **INNER JOIN**.
2. Retrieve all customers including those without orders using **LEFT JOIN**.
3. Retrieve all orders including those with missing customer details using **RIGHT JOIN**.
4. Retrieve complete list of customers and orders using **FULL OUTER JOIN**.

5. Program / SQL

5.1 Creating Base Tables

```
CREATE TABLE Customer (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(100),  
    Email VARCHAR(100)  
);  
  
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE,  
    Amount DECIMAL(10,2),  
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)  
);
```

5.2 INNER JOIN

Retrieve customers who have placed orders.

```
SELECT  
    C.CustomerID, C.CustomerName,  
    O.OrderID, O.OrderDate, O.Amount  
FROM Customer C  
INNER JOIN Orders O  
ON C.CustomerID = O.CustomerID;
```

5.3 LEFT JOIN

Retrieve all customers including those with no orders.

```
SELECT
    C.CustomerID, C.CustomerName,
    O.OrderID, O.OrderDate, O.Amount
FROM Customer C
LEFT JOIN Orders O
ON C.CustomerID = O.CustomerID;
```

5.4 RIGHT JOIN

Retrieve all orders, including those that have no matching customer.

```
SELECT
    C.CustomerID, C.CustomerName,
    O.OrderID, O.OrderDate, O.Amount
FROM Customer C
RIGHT JOIN Orders O
ON C.CustomerID = O.CustomerID;
```

5.5 FULL OUTER JOIN

Retrieve all data, whether matched or unmatched.

Note:MySQL does not support FULL OUTER JOIN directly; we use UNION.

```
SELECT
    C.CustomerID, C.CustomerName,
    O.OrderID, O.OrderDate, O.Amount
FROM Customer C
```

```
LEFT JOIN Orders O
ON C.CustomerID = O.CustomerID

UNION

SELECT
    C.CustomerID, C.CustomerName,
    O.OrderID, O.OrderDate, O.Amount
FROM Customer C
RIGHT JOIN Orders O
ON C.CustomerID = O.CustomerID;
```

6. Output

- Successfully retrieved order details using INNER JOIN.
- Retrieved all customers irrespective of order status using LEFT JOIN.
- Retrieved all orders including unmatched records using RIGHT JOIN.
- Displayed complete combined records using FULL OUTER JOIN.

7. Conclusion

This experiment demonstrated the practical usage of SQL joins to retrieve related records from multiple tables. It enhanced understanding of combining data using INNER, LEFT, RIGHT, and FULL OUTER JOIN operations.

Assignment-5

Title: Design and Implement a Sales Management System with Stored Procedures and Functions for Efficient Data Handling.

1. Aim

To design and implement PL/SQL stored procedures and functions for efficient data handling in a Sales Management System.

2. Objectives

- To understand the creation and execution of PL/SQL stored procedures.
- To implement user-defined functions for performing calculations.
- To automate data insertion and computation using PL/SQL blocks.

3. Theory

A **stored procedure** is a named PL/SQL block stored in the database, used to perform a specific task such as inserting or updating data. It improves reusability, security, and performance.

A **function** is similar to a procedure but must return a value. Functions are widely used for calculations like totals, averages, and financial computations.

Typical components:

- **IN / OUT parameters**
- **EXCEPTION handling**
- **RETURN** statement (for functions)

In a Sales Management System, procedures can insert new sales records, and functions can compute totals or summaries.

4.Tasks

1. Create a stored procedure to insert new sales records into the `Sales` table.
2. Create a function to calculate and return the total sales amount from the `Sales` table.
3. Execute the procedure and function with test inputs.

5. Program / PL-SQL Code

(A) Create Sales Table

```
CREATE TABLE Sales (  
    sale_id NUMBER PRIMARY KEY,  
    customer_name VARCHAR2(50),  
    amount NUMBER(10,2),  
    sale_date DATE  
);
```

(B) Stored Procedure to Add Sales Data

```
CREATE OR REPLACE PROCEDURE add_sale  
(  
    p_sale_id      IN NUMBER,  
    p_customer_name IN VARCHAR2,
```

```
        p_amount          IN NUMBER
    )
AS
BEGIN
    INSERT INTO Sales(sale_id, customer_name, amount, sale_date)
    VALUES(p_sale_id, p_customer_name, p_amount, SYSDATE);

    DBMS_OUTPUT.PUT_LINE('Sale record inserted successfully.');
```

END;

/

(C) Function to Calculate Total Sales

```
CREATE OR REPLACE FUNCTION get_total_sales
RETURN NUMBER
AS
    total NUMBER;
BEGIN
    SELECT SUM(amount) INTO total FROM Sales;
    RETURN total;
END;
```

/

(D) Execute Procedure and Function

```
BEGIN
    add_sale(101, 'Rahul Sharma', 4500);
    add_sale(102, 'Neha Verma', 3200);
END;
```

/

```
DECLARE
    total_sales NUMBER;
BEGIN
    total_sales := get_total_sales();
    DBMS_OUTPUT.PUT_LINE('Total Sales = ' || total_sales);
```

```
END;
```

```
/
```

6. Output

- Sales records were successfully inserted using the stored procedure.
- The function correctly returned the total sales amount.
Result: PL/SQL procedures and functions were successfully implemented for a Sales Management System.

Conclusion

The experiment successfully implemented PL/SQL procedures and functions. Sales data was inserted efficiently, and total sales were calculated correctly, demonstrating improved automation and data handling.
